

BYTERIDE FINANCE ASSISTANT

Natural-language questions over a financial ledger — every answer computed by SQL against real records, never guessed by a model.

BVP TECH CATALYST HACKATHON • 5-PERSON TEAM

LIVE DEMO <https://byteride-finance-assistant.onrender.com>

Trustworthy answers over real transaction data

- Finance teams and individuals ask plain-English questions of raw transaction data: “where did I spend the most”, “what's my tax total”.
- A generic LLM answers by producing a plausible-looking number — no SQL, no evidence, no way to check it.
- In finance, a confident wrong number is worse than no answer at all.
- What's needed: an assistant that computes with real records and shows its work — or admits, honestly, when the data can't answer.

✗ A GENERIC LLM

“You spent about ₹15,90,000 last month.”

No SQL. No evidence. No way to verify it.

✓ BYTERIDE

“Total for 2026-05-01 to 2026-05-31: ₹1,62,50,89,107.”

+ the exact SQL, 25 masked evidence rows, a confidence badge, and anomaly flags — a real recorded answer.

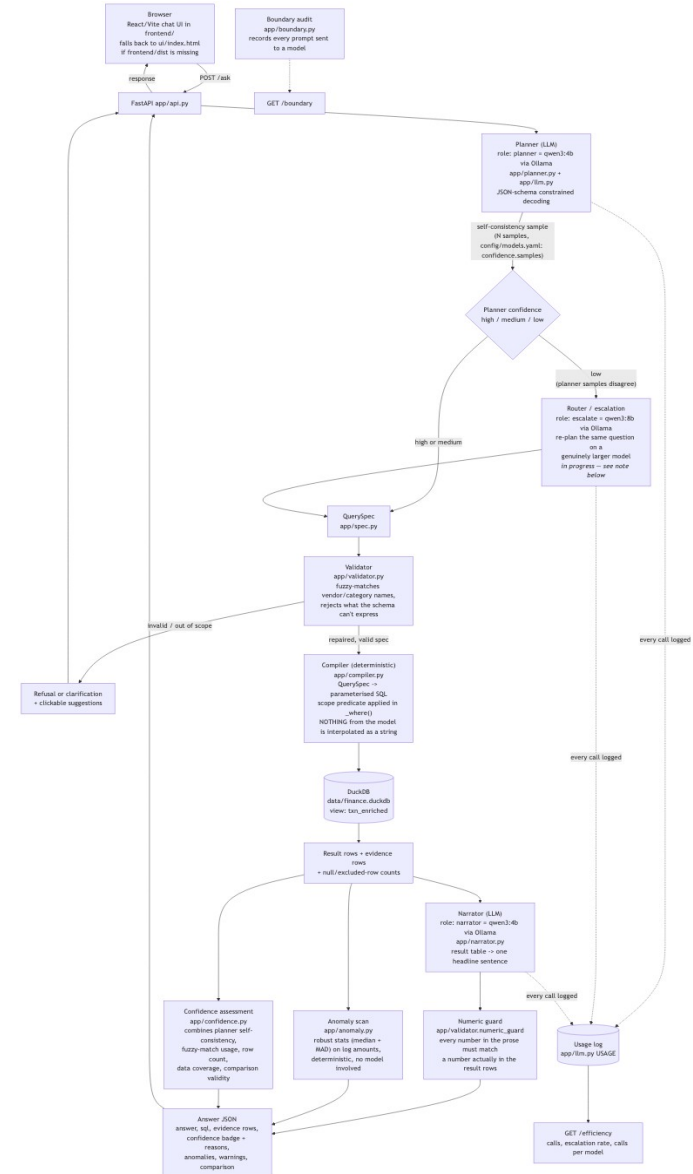
APPROACH

One picture: the request path

- question → planner (small LLM) → QuerySpec
- QuerySpec → validator → compiler → parameterised SQL
- SQL → DuckDB → result + evidence rows
- result → narrator (small LLM) → numeric guard

THE ONE RULE

The LLM never does arithmetic. SQL is compiled from a validated spec — the model never sees a data row.



What makes an answer verifiable

1

SQL shown with every answer

The exact parameterised SQL that produced the number ships alongside it.

2

Evidence rows

Up to 25 real, PII-masked rows returned with every aggregate — the answer is checkable, not asserted.

3

Confidence badge

High / medium / low, from planner self-consistency, fuzzy-match usage, row count and comparison validity.

4

Anomaly flags

Robust statistics (median + MAD on log amounts) flag outliers deterministically — no model involved.

5

Honest refusals

“This dataset has no reconciliation status ... I will not infer one.” Declines instead of guessing.

6

Boundary audit

GET /boundary logs every prompt sent to a model. Rule: the model receives the question and the schema, never a data row.

7

PII masked in ETL

account_number → last 4 digits;
utr_number → [redacted]. Masking happens in SQL — raw values never leave the database.

Data & scope decisions

Seven calls the real schema forced — each one a deliberate choice, not a lookup.

1

Reconciliation — dropped

Not in the data. The assistant must say so instead of inventing a status.

2

Scope: personal spend analytics

Vendor / category / channel / bank / period breakdowns, comparisons, lookups. Budgets, forecasts and chart-of-accounts are refused.

3

Categories are derived

app/enrich.py classifies each narration at load time (TAX, SALARY, RENT, ..., UNCATEGORISED) — never stored, never authoritative.

4

Vendors parsed from free text

No vendor table. Counterparty is parsed from the narration at load time. Open issue: branch/location noise still splits some vendors in two.

5

Encryption: ETL only

AES-256 decrypt happens once at load, never per query. account_number masked to last 4; utr_number never decrypted.

6

DuckDB over Snowflake

Decided: identical demo, but no venue-wifi dependency and no cold-warehouse pause at the worst moment.

7

Negative balances — open

available_balance runs to -₹13,16,29,423 on several accounts. Sign convention or export issue — asked the organisers.

Why small local models, and when to escalate

WHY SMALL & LOCAL

- Privacy — the model receives only the question and the schema, never a data row (app/boundary.py, audited).
- Cost — no per-token bill for local inference against a self-hosted Ollama server.
- Latency — no network hop; the whole pipeline answers in ~1.5-2s end to end.

HOSTED VARIANT (LIVE ON RENDER)

gemini-3.5-flash-lite for planner / narrator / router; **gemini-3.6-flash** as the cloud escalation tier — same config/models.yaml roles, same pipeline.

DEFAULT TIER

qwen3:4b — planner, narrator, router

Every question starts here. Deterministic sampling (temperature 0, seed 42) so the same question gets the same spec.

ESCALATION TIER

qwen3:8b — a genuinely larger model

Escalating to the same model proves nothing — 8b must differ from the 4b planner for the rate to mean anything.

WHEN ESCALATION TRIGGERS

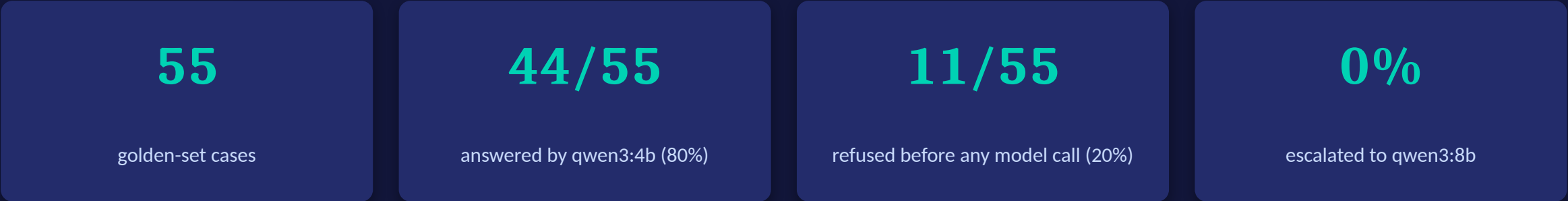
- Planner self-consistency agreement falls below the confidence threshold, or
- the validator's repair loop fails a second time.

Model comparison — golden set (55 cases)

Model	Params / size	Overall accuracy	Refusal correctness	Latency p50	Latency max
qwen3:1.7b	2.0B / 1.36 GB	54/55 (98%)	11/11 (100%)	458 ms	3,546 ms
qwen3:4b — default	4.0B / 2.5 GB	55/55 (100%)	11/11 (100%)	1,517 ms	7,502 ms
qwen3:8b — escalation	8.2B / 5.23 GB	54/55 (98%)	11/11 (100%)	2,014 ms	29,013 ms
tiered 4b→8b (samples=1)	4b + 8b co-resident	55/55 (100%)	11/11 (100%)	1,561 ms	7,600 ms
tiered 4b→8b (samples=3)	4b + 8b co-resident	55/55 (100%)	11/11 (100%)	1,533 ms	2,753 ms

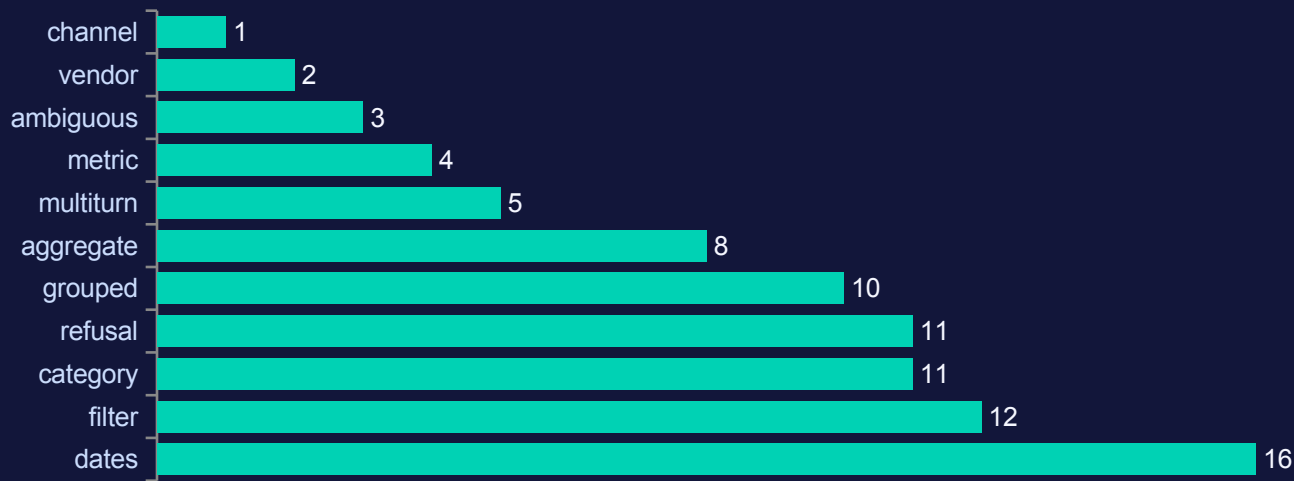
qwen3:4b is the pick: the only raw model at a clean 55/55, roughly 3.3× faster p50 than qwen3:8b. The escalation tier **never engaged** on this golden set — 0 escalations in both tiered runs. qwen3:8b is not better here: it scored worse than 4b (54/55) while being both the largest and by far the slowest model (29s worst case).

Which model answered what



Tiered (default config) latency: p50 1,561 ms · max 7,600 ms

GOLDEN SET COMPOSITION (TAGS, 55 CASES)



KNOWN GAP

The self-consistency escalation path (plan_with_confidence) is exercised by /ask, not by this eval harness — so a 0% escalation rate here means “untested”, not “never needed”.

Golden-set results, per bucket

Bucket	Total	1.7b	4b	8b	Tiered
aggregate	8	8/8	8/8	8/8	8/8
ambiguous	3	3/3	3/3	3/3	3/3
category	11	11/11	11/11	11/11	11/11
channel	1	1/1	1/1	1/1	1/1
dates	16	16/16	16/16	15/16	16/16
filter	12	12/12	12/12	12/12	12/12
grouped	10	9/10	10/10	10/10	10/10
metric	4	3/4	4/4	4/4	4/4
multiturn	5	5/5	5/5	5/5	5/5
refusal	11	11/11	11/11	11/11	11/11
vendor	2	2/2	2/2	2/2	2/2

Refusal correctness: **11/11 (100%)** across every configuration tested — including the two single misses shown in red (1.7b on a grouping spec, 8b on a date-range spec).

Six steps to the money shot

1

“Which vendors did I pay the most last month?”

EMI BAJAJ FINANCE is highest at ₹13,73,25,576. Followed by LIC PREMIUM POLICY at ₹3,03,80,709.

2

Expand the SQL + evidence panel

The exact parameterised SQL, plus masked evidence rows (account_number → XXXXXX1922, utr_number → [redacted]).

3

“How does that compare with the month before?” (same session)

The biggest move is BIG BAZAAR RETAIL, up ₹1,13,26,049 — no context repeated in the follow-up question.

4

“How much tax did I pay this year?”

Total for 2026-01-01 to 2026-12-31: ₹1,95,63,10,950 — a derived category, shown as derived.

5

“Which transactions are unreconciled?” — the money shot

“This dataset has no reconciliation status ... I will not infer one.” Refuses cleanly in 0.007s.

6

GET /efficiency

Which model answered, and the escalation rate — the model-efficiency claim, live.

What's next — known limitations

Vendor name splitting

Branch/location noise (“X” vs “X DAHISAR EAST”) still corrupts some “where did I spend most” answers — needs a real place-name list, not ten sample rows.

Negative balances — open question

available_balance runs to -₹13,16,29,423 on several accounts. Sign convention or export issue, unresolved — asked the organisers.

TAX rules need the real corpus

Keyword rules are tuned against a handful of examples; must be re-validated once the organisers' full export lands.

Amount-unit parsing gaps

The planner doesn't resolve “lakh” as a unit, and combining an amount filter with a date filter can silently drop the date — found during real Q&A testing.

UNCATEGORISED coverage unreported

The UNCATEGORISED percentage isn't yet surfaced in the pipeline — if it's high, spend-by-category answers understate the gap.

Escalation path untested by evals

Self-consistency escalation is exercised by /ask but not by make eval — golden-set escalation rate should be read as “untested”, not “zero risk”.

TRY IT

Repo, live demo, and how to run it

REPO

`github.com/Aahan215/
ByteRideFinanceAssistant`

LIVE DEMO

`byteride-finance-assistant.onrender.com`

Render free tier — cold start up to ~1 min, hosted Gemini, synthetic 200k-row dataset.

RUN IT LOCALLY — THREE COMMANDS

```
$ make setup  
$ make load  
$ make ui-install && make ui-build && make run  
  
→ open http://localhost:8765
```